```
  def destroy
    @user.destroy

    respond_to do |format|
      format.html { redirect_to users_url,
        notice: "User #{@user.name} deleted" }
      format.json { head :no_content }
    end
  end

» rescue_from 'User::Error' do |exception|
»   redirect_to users_url, notice: exception.message
» end
```

This code still has a potential timing issue: it's still possible for two administrators each to delete the last two users if their timing is right. Fixing this would require more database wizardry than we have space for here.

In fact, the login system described in this chapter is rudimentary. Most applications these days use a plugin to do this.

A number of plugins are available that provide ready-made solutions that not only are more comprehensive than the authentication logic shown here but generally require less code and effort on your part to use. Devise[72] is a common and popular gem that does this.

What We Just Did

By the end of this iteration, we've done the following:

- We used `has_secure_password` to store an encrypted version of the password into the database.
- We controlled access to the administration functions using before action callbacks to invoke an `authorize` method.
- We used `rails console` to interact directly with a model (and dig us out of a hole after we deleted the last user).